P06 Player Animation

CRE132 worksheet

What you'll make

A platformer hero that looks alive: standing still it plays its idle, moving it runs, going up it shows the jump pose, coming down it puts its arms up, and it does a quick crouch when it lands. It also turns to face the way it's moving.

The movement already works. You'll build the **Animator Controller** that chooses between the animations, then write one short script that tells the Animator what the player is doing. By the end you'll know how **parameters**, **transitions** and **Any State** work together, and why the movement script and the animation script are kept apart.

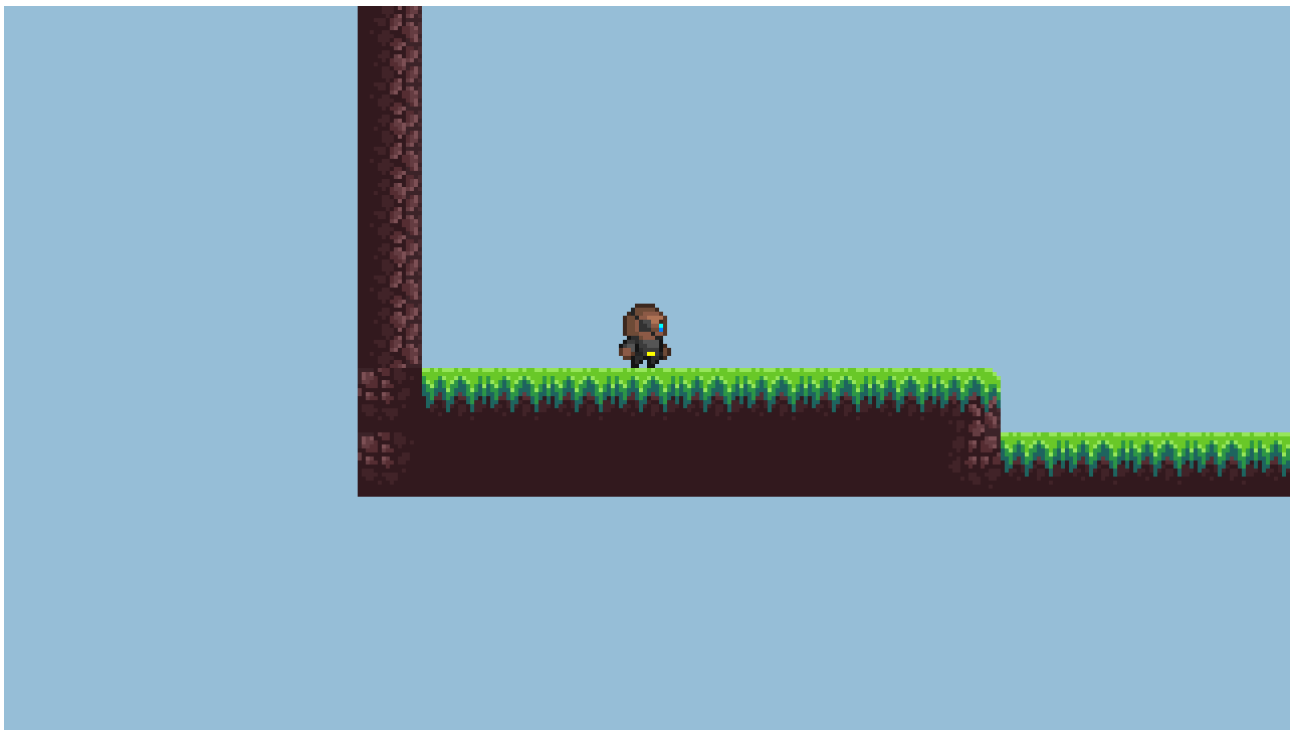


Figure 1: The finished scene: the hero mid-jump between the platforms.

Before you start

1. Unzip `CRE132_P06_PlayerAnimation_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P06_PlayerAnimation_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P06_PlayerAnimation`.
3. Press **Play**. You can move (A/D or arrows) and jump (Space), but the hero is a still picture that always faces right. Press Play again to stop.

The **Player** has the `BetterJumpController` from P02 for movement and an **Animator** component, but the Animator's **Controller** slot is empty. The clips are ready in `Assets/_game/Animation/Clips/ElvGames`. You'll build the controller, then write the script that tells it what the player is doing.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The animation clips

In the Project window, open `Assets/_game/Animation/Clips/ElvGames`. There's a clip for each pose: **Hero_Idle**, **Hero_Run**, **Hero_Jump**, **Hero_Fall** and **Hero_Land** (and **Hero_Dead**, which you'll use in P10). Click one and press Play in the Inspector's preview to watch it.

- **The art pack has no run or landing sheet. Hero_Run** is the walk played faster (14 frames a second), and **Hero_Land** reuses two crouched frames from the jump. Reusing frames like this is normal with small asset packs.
- **Hero_Land doesn't loop**: it plays once and stops. The others loop.

2. The Animator Controller

You built a controller with two states in C07. This one has five.

1. In the Project window, go to `Assets/_game/Animation/Controllers`. Right-click and choose **Create > Animation > Animator Controller**. Name it `HeroPlatformer`.
2. Select the **Player** and drag `HeroPlatformer` into the **Controller** slot of its **Animator** component.
3. Open **Window > Animation > Animator**. It shows `HeroPlatformer`, empty apart from **Entry**, **Any State** and **Exit**.
4. Drag **Hero_Idle** from `Clips/ElvGames` into the Animator window first, then **Hero_Run**, **Hero_Jump**, **Hero_Fall** and **Hero_Land**. Each becomes a grey box, a **state**. The first one is orange because it's the **default state**, the one that plays first. Select each state and rename it at the top of the Inspector: **Idle**, **Run**, **Jump**, **Fall** and **Land**.

Parameters. These are what the Animator needs to know about the player. On the **Parameters** tab at the left of the Animator window, click **+** and add:

- **Speed** (**Float**): how fast it's moving sideways.
- **VerticalSpeed** (**Float**): how fast it's moving up (positive) or down (negative).
- **IsGrounded** (**Bool**): whether it's standing on something.

Transitions. Make each arrow in the table below: right-click the **From** state (or **Any State**), choose **Make Transition** and click the **To** state. Then select the arrow and, in the Inspector:

1. Untick **Has Exit Time** (except on **Land to Idle**).
2. Open **Settings** and set **Transition Duration** to **0**.
3. On the two arrows from **Any State**, also untick **Can Transition To Self** (it's under **Settings** too).
4. Under **Conditions**, click **+** once for each condition and set it as in the table. A Bool condition shows **true** or **false** instead of a number.

From	To	Conditions
Idle	Run	Speed Greater 0.1
Run	Idle	Speed Less 0.1
Any State	Jump	IsGrounded false, VerticalSpeed Greater 0.1
Any State	Fall	IsGrounded false, VerticalSpeed Less −0.1
Jump	Idle	IsGrounded true
Fall	Land	IsGrounded true
Land	Run	Speed Greater 0.1
Land	Idle	none: leave **Has Exit Time** ticked and set **Exit Time** to **1**, so it waits for the clip to finish

(The Finished controller also has a **Die** trigger and a **Dead** state, **Any State > Dead**, for P10. You don't need them yet.)

- **Any State** means "from whichever state we're in". You can leave the ground from Idle, Run or Land, so one arrow from Any State is simpler than three.
- **Can Transition To Self** is unticked on the Any State arrows. If it were ticked, Jump would restart itself every frame while the conditions are true and you'd only ever see its first frame.
- **0.1 instead of 0**: a standing player still has tiny velocities (0.0001) from the physics. A small threshold stops it flickering between states.
- **No blending for pixel art:** pixel art can't blend between frames, so every transition has **Has Exit Time** off (except Land to Idle) and a **Transition Duration** of 0, to change straight away.

Test it: press Play. The hero now breathes (its idle), but never runs or jumps: nothing sets the parameters yet, which is the next part.

3. The PlatformerAnimator script

Make a folder `Assets/_game/Scripts/P06_PlayerAnimation` and create a script in it called `PlatformerAnimator`:

```
// Sends the player's movement to the Animator: Speed (left/right), VerticalSpeed
// (up/down) and IsGrounded.
// The Animator Controller's transitions use them to pick Idle, Run, Jump, Fall or Land.
using UnityEngine;

public class PlatformerAnimator : MonoBehaviour
{
    [SerializeField] private float turnThreshold = 0.1f;

    private Rigidbody2D body;
    private Animator animator;
    private SpriteRenderer spriteRenderer;
    private BetterJumpController controller;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
        spriteRenderer = GetComponent<SpriteRenderer>();
        controller = GetComponent<BetterJumpController>();
    }
```

```
    private void Update()
    {
        Vector2 velocity = body.linearVelocity;

        animator.SetFloat("Speed", Mathf.Abs(velocity.x));
        animator.SetFloat("VerticalSpeed", velocity.y);
        animator.SetBool("IsGrounded", controller.IsGrounded);

        if (velocity.x < -turnThreshold)
        {
            spriteRenderer.flipX = true;
        }
        else if (velocity.x > turnThreshold)
        {
            spriteRenderer.flipX = false;
        }
    }
}
```

- **It reads, it doesn't move.** The script looks at the Rigidbody2D's velocity and the controller's public **`IsGrounded`**, and passes them on. `BetterJumpController` moves the player and knows nothing about animation.
- **`SetFloat` and `SetBool`** set the parameters by name. The names must match the Parameters tab exactly, capital letters included.
- **`Mathf.Abs`** turns a negative speed (moving left) into a positive one, so Idle to Run works in both directions.
- **Flipping:** moving left flips the sprite. When the player stops, neither `if` is true, so it keeps facing the way it was going.

Add it: select the **Player** and click **Add Component > Platformer Animator**. Leave **Turn Threshold** at **0.1**.

Test it: press Play. Run left and right, jump onto a platform and drop off it. The hero should run, jump, fall and land, and face the way it moves.

4. Watch it work

Keep the **Animator** window open (dock it next to the Game view) and select the **Player** while the game is playing. The state that's playing has a blue progress bar, and the **Parameters** tab shows the values changing as you move.

Test it: jump and watch the path: Idle, Jump, Fall, Land, Idle. Make a tiny hop and you may see Jump go straight back to Idle without falling: that's the Jump to Idle arrow.

Check your work

- ☐ `HeroPlatformer` has the five states, three parameters and eight transitions from the table, with **Idle** as the default.
- ☐ Standing still plays Idle; moving plays Run.
- ☐ Going up shows the jump pose, coming down shows the fall pose.
- ☐ Landing shows a quick crouch, then Idle or Run.
- ☐ The hero faces the way it moves and keeps facing that way when it stops.
- ☐ Compare with `CRE132_P06_PlayerAnimation_Finished` if something behaves differently.

Key points

- **Movement and animation are separate scripts.** The movement script moves the player; the animator script only *shows* what's happening. Neither knows about the other's details.
- **Parameters are the Animator's inputs.** Code sets them, and transitions read them to choose a state.
- **Any State** transitions suit things that can happen at any time, like falling or getting hurt.
- **Use small thresholds** (0.1, not 0) so tiny velocities don't make the animation flicker.

If something goes wrong

- **The hero stays a still picture:** the **Controller** slot on the Player's **Animator** is empty. Drag `HeroPlatformer` into it.
- **The hero starts in the wrong pose: Idle** isn't the default state. Right-click it and choose **Set as Layer Default State**.
- **"The type or namespace name 'BetterJumpController' could not be found":** you're in the wrong project, or another script has an error. Fix the first red error in the Console first.
- **"Parameter 'speed' does not exist" in the Console:** a parameter name in your code is spelled differently from the Animator's. `Speed`, `VerticalSpeed` and `IsGrounded` are case-sensitive.
- **The jump or fall shows only one frame:** an Any State arrow has **Can Transition To Self** ticked.
- **The hero stays in the jump or fall pose after landing:** the **Jump to Idle** or **Fall to Land** arrow is missing, or its condition isn't **IsGrounded true**.
- **The hero crouches for ever after landing:** the **Land to Idle** arrow needs **Has Exit Time** ticked.
- **Changes of pose come late or slide:** an arrow still has **Has Exit Time** ticked, or its **Transition Duration** isn't 0.
- **The hero flickers between Idle and Run when standing still:** a threshold has been changed to 0. Put it back to 0.1.
- **NullReferenceException in PlatformerAnimator:** the script is on an object with no Animator, Rigidbody 2D or BetterJumpController. It belongs on the **Player**.

Try it

1. Set **Has Exit Time** on the Idle to Run arrow and play. Why does running now feel slow to start?
2. Tick **Can Transition To Self** on **Any State > Fall**, play and drop off a ledge. What happens to the fall animation? Untick it again.
3. Add a **Hurt** trigger parameter and a Hurt state. Copy `Health` in from your S09 project, put it on the Player, and in its **On Damaged** event call the Animator's **SetTrigger** with `Hurt`.
4. Use the Dash from P03 and add a Dash state.
5. Swap Run for a 1D blend tree (C12) so the hero walks when the stick is only half pushed.